Parse Trees

Programming Languages Intro

Dr. Igor Steinmacher

From last topic

- BNF Grammar
  - metalanguage used to define the syntax of a language
- Language
  - All strings that can be derived from a grammar (S)
- Derivation
  - Process from the start symbol until the string

Parse Trees

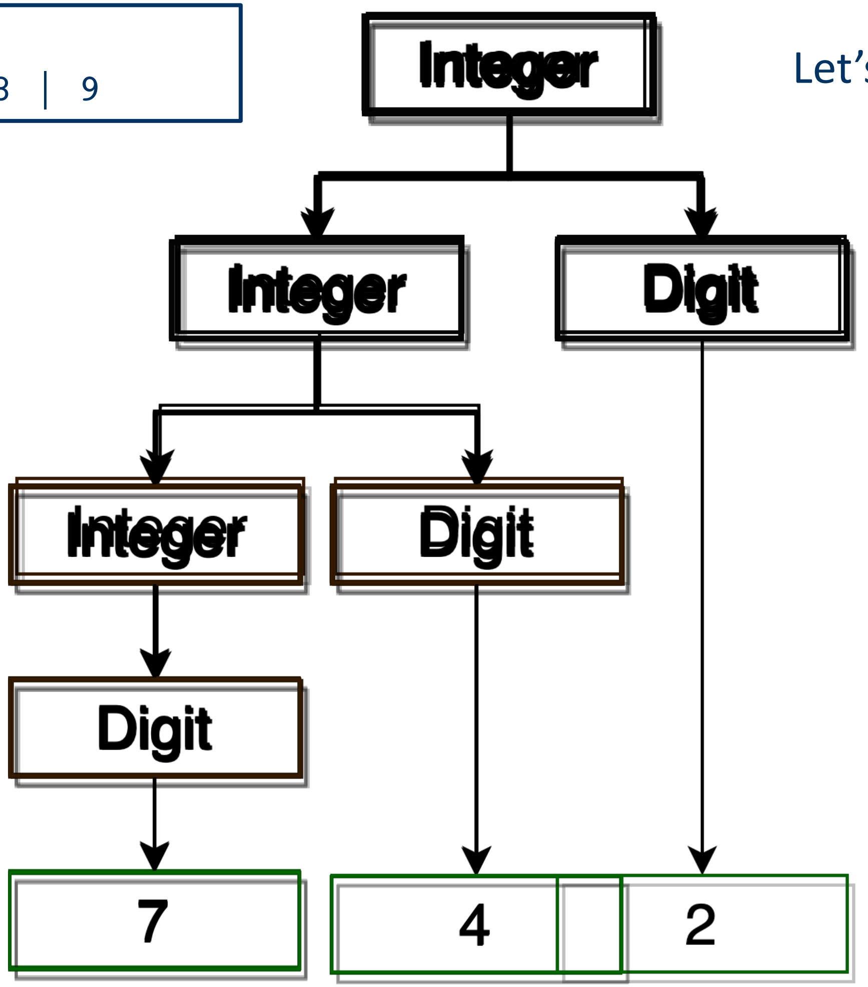
- A *parse tree* is a graphical (*hierarchical*) representation of a derivation
  - **root** → start symbol
  - **internal node** → non-terminal nodes (step of derivation)
  - **child of a node** → right-hand side of a production
  - **leaf node** → terminal symbols of the derived string, reading from left to right.

Parse Tree

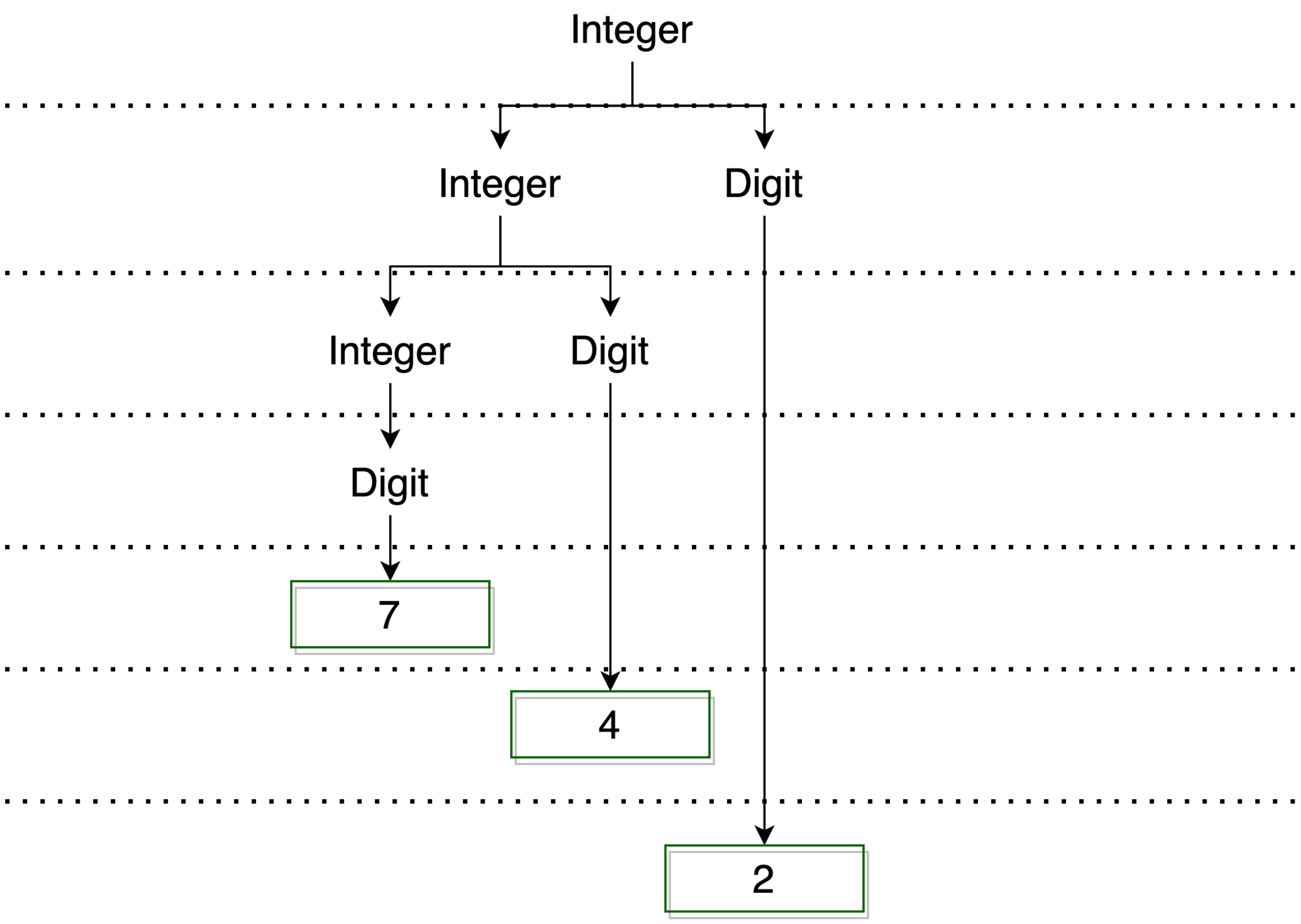
```
Integer → Digit | Integer Digit
Digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Let's derive **742** again

root → start symbol
internal node → non-terminal nodes (step of derivation)
child of a node → right-hand side of a production
leaf node → terminal symbols of the derived string, reading from left to right.



BNF: Derivation (leftmost)



Arithmetic Expression Grammar

- Arithmetic expressions with 1-digit integers, addition, and subtraction.

$$\mathtt{Expr} \rightarrow \mathtt{Expr} + \mathtt{Term} \mid \mathtt{Expr} - \mathtt{Term} \mid \mathtt{Term}$$

$$\mathtt{Term} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

- How would a compiler parse

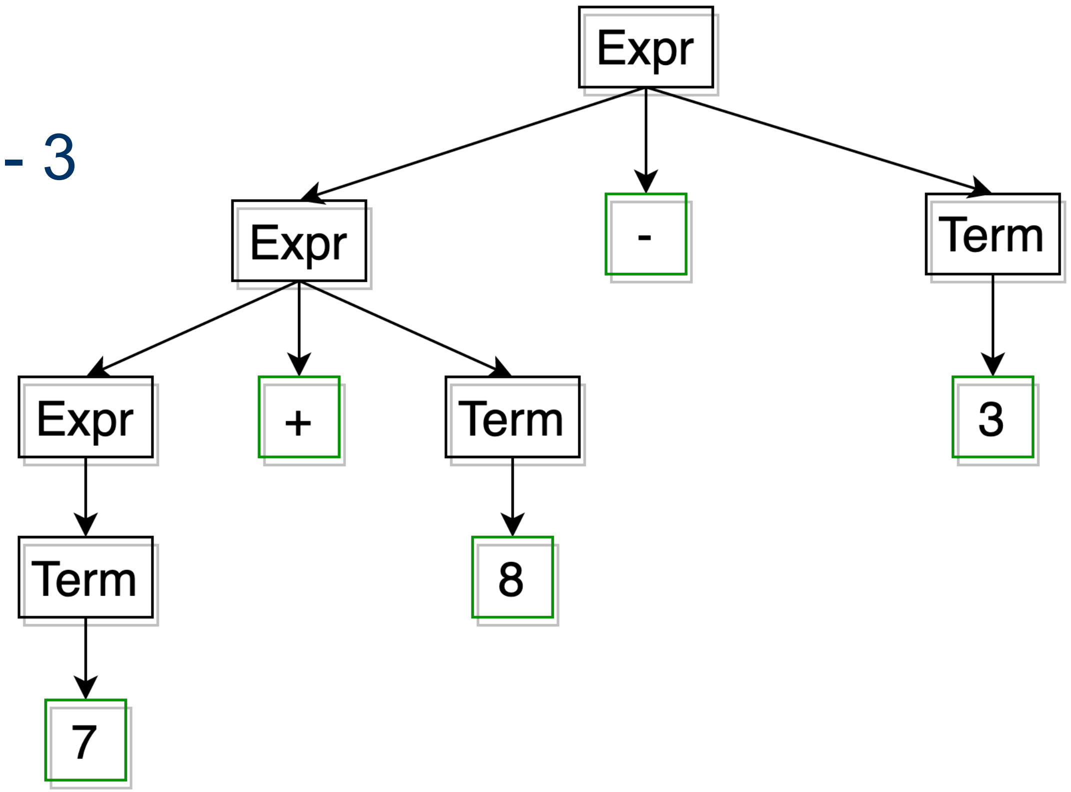
7 + 8 - 3

Arithmetic Expression Grammar

`Expr → Expr + Term | Expr – Term | Term`

`Term → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`

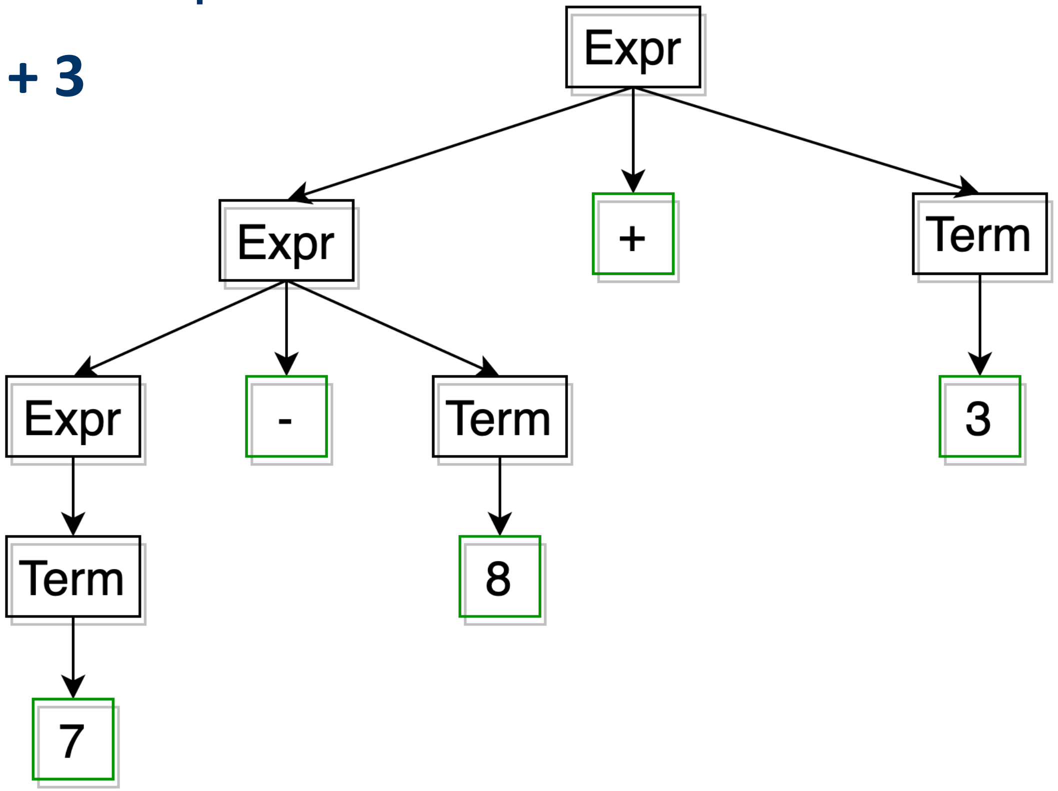
7 + 8 - 3



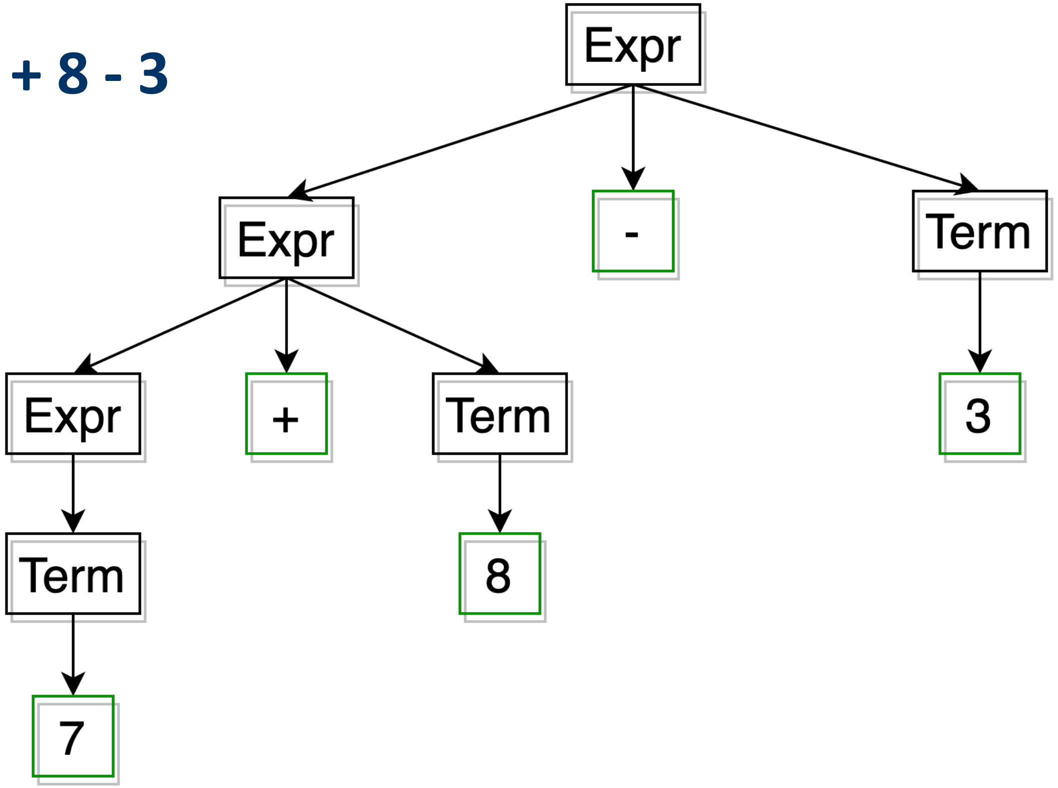
Arithmetic Expression Grammar

- Left-associative recursion
  - associativity → + and - have the same precedence

7 - 8 + 3



7 + 8 - 3

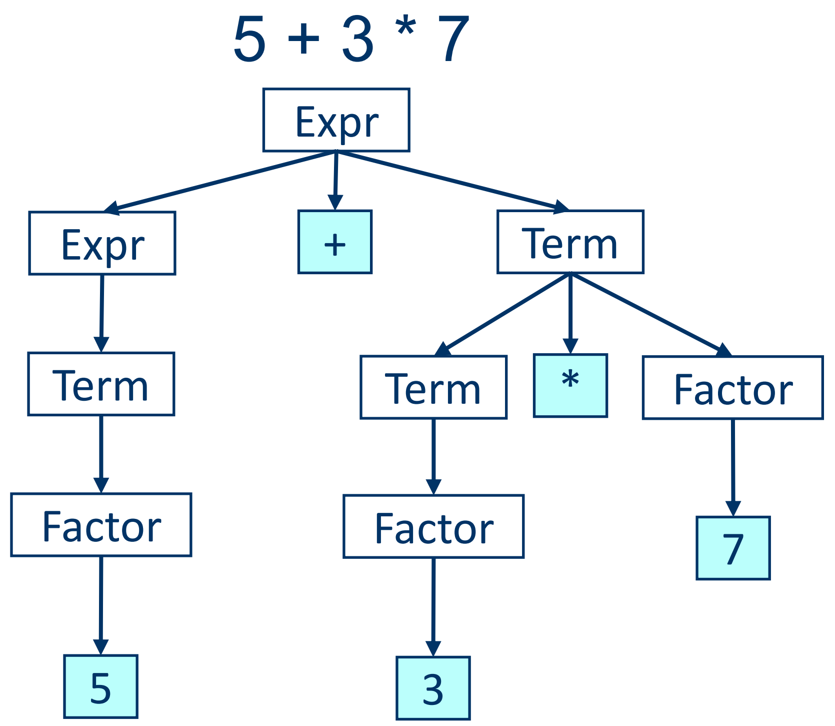


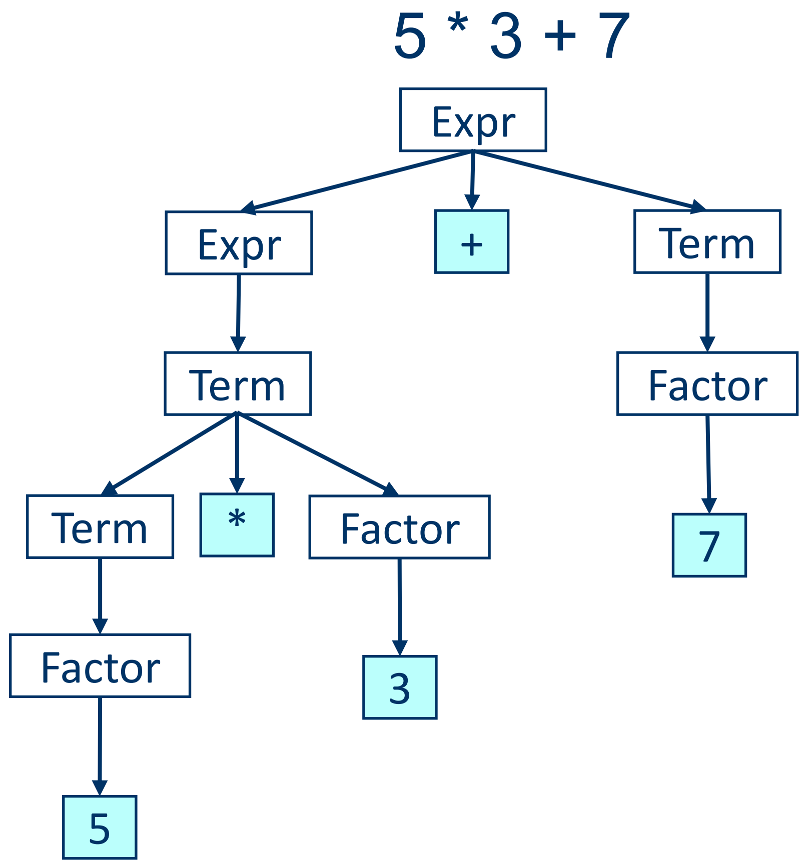
Precedence and Associativity

- Precedence
  - Order in which operations are performed within expressions
- Associativity
  - Order for operations with the same precedence
  - Operations can be left- or right-associative
- + and - are left-associative operators in mathematics;
- * and / have higher precedence than + and -

Precedence

```
Expr    → Expr + Term | Expr – Term | Term
Term    → Term * Factor | Term / Factor | Term % Factor | Factor
Factor  → Primary ** Factor | Primary
Primary → 0 | ... | 9 | ( Expr )
```

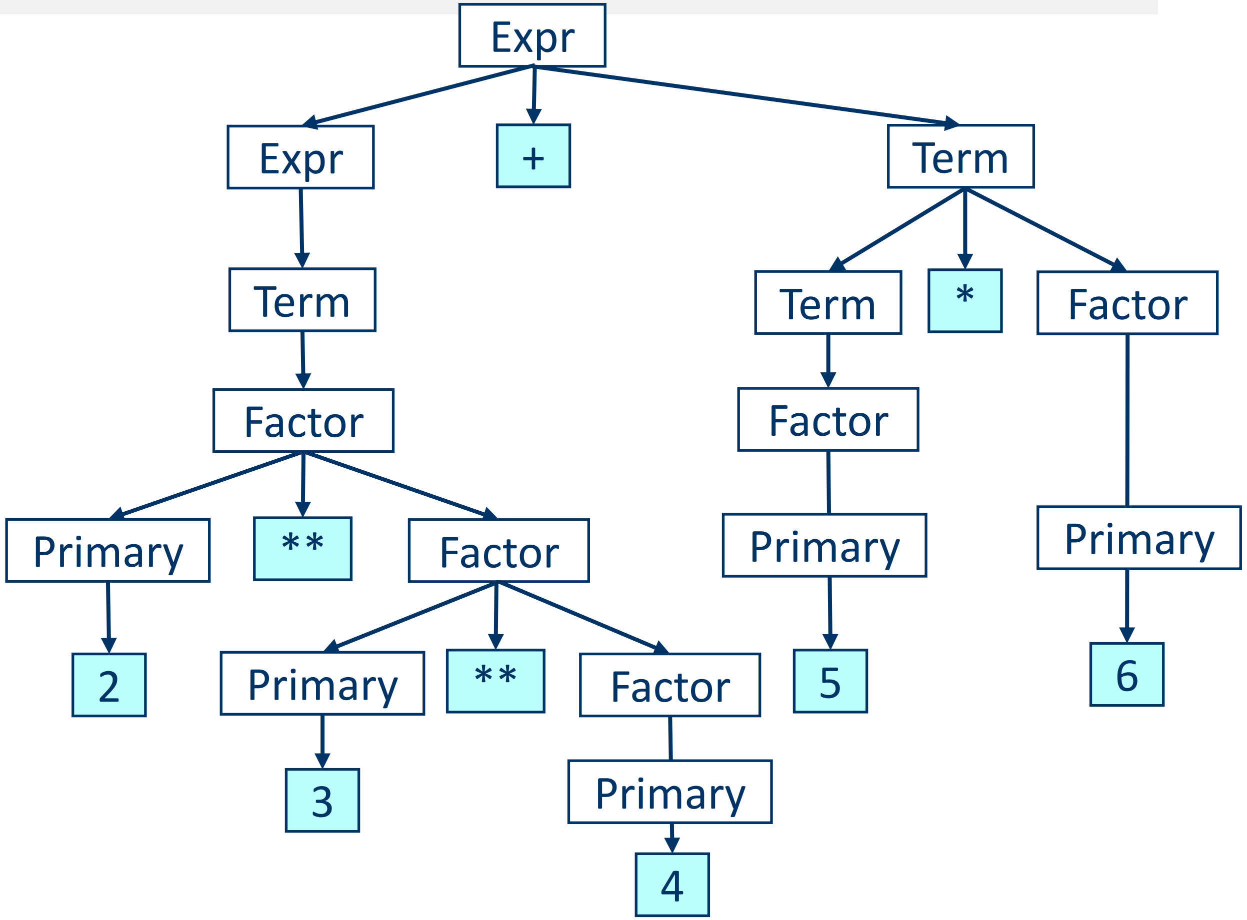




Precedence

```
Expr    → Expr + Term | Expr – Term | Term
Term    → Term * Factor | Term / Factor | Term % Factor | Factor
Factor  → Primary ** Factor | Primary
Primary → 0 | ... | 9 | ( Expr )
```

2**3**4+5*6



Associativity and Precedence for our grammar

Precedence	Associativity	Operators
3	Right to left	**
2	Left to right	* / %
1	Left to right	- +

prec 3 2 ** 3 ** 4 + 5 * 6

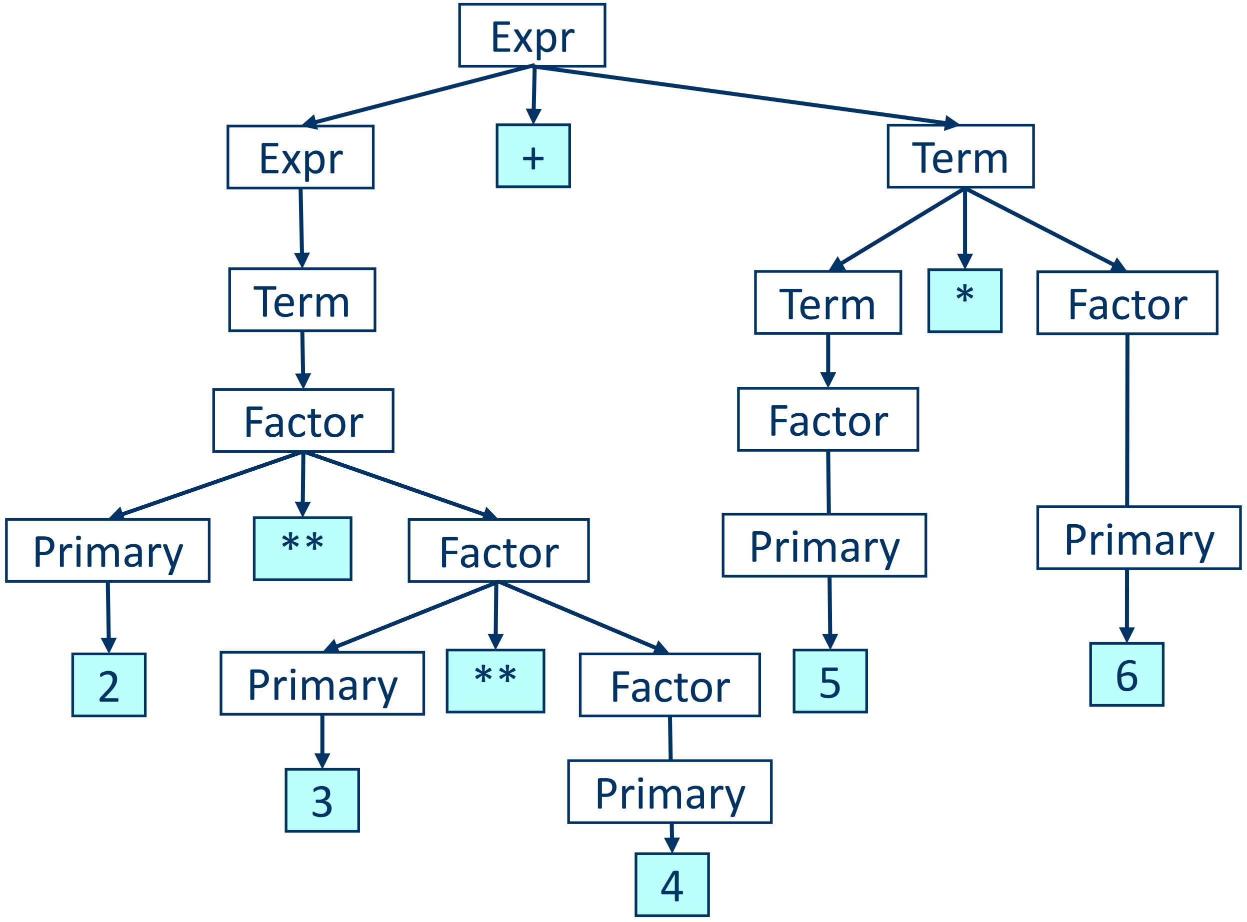
right 2 ** (3 ** 4) + 5 * 6

(2 ** (3 ** 4)) + 5 * 6

prec 2 (2 ** (3 ** 4)) + 5 * 6

(2 ** (3 ** 4)) + (5 * 6)

prec 1 (2 ** (3 ** 4)) + (5 * 6)



See you in our next Lesson!